Analysis and Design of Algorithms

NUS National University of Singapore

CS3230

Reasoning About Algorithms

- Invariant and Iterative algorithms
- Recursive algorithms

Prof. Lee Wee Sun

Insertion Sort

INSERTION-SORT(A)

```
for j = 2 to A.length
    key = A[j]
    // Insert A[j] into sorted seq A[1 .. j-1]
    i = j – 1
    while i > 0 and A[i] > key
        A[i+1] = A[i]
        i = i – 1
    A[i+1] = key
```

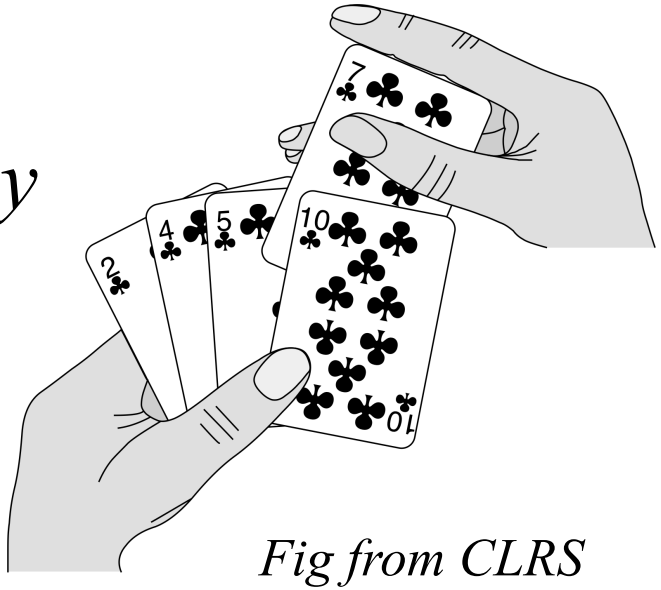

Fig from CLRS

Correctness of Iterative Algorithms

The key step in the reasoning about the correctness of iterative algorithms is the invention of a condition called the ***loop invariant***, which is supposed to be

- true at the beginning of an iteration, and
- remains true at the beginning of the next iteration

In insertion sort, the **loop invariant** is:

At the start of each of the iterations of the **for** loop (line 1), the subarray $A[1 .. j\text{-}1]$ consists of the elements originally in $A[1 .. j\text{-}1]$, but in sorted order

```
INSERTION-SORT(A)
1.  for j = 2 to A.length
2.      key = A[j]
3.      // Insert A[j] into sorted seq A[1 .. j-1]
4.      i = j – 1
5.      while i > 0 and A[i] > key
6.          A[i+1] = A[i]
7.          i = i – 1
8.      A[i+1] = key
```

Using Invariant

To understand the correctness of an algorithm using an invariant, we show three things:

- **Initialization:** The invariant is true before the first iteration of the loop
- **Maintenance:** If the invariant is true before an iteration, it remains true before the next iteration
- **Termination:** When the algorithm terminates, the invariant provides a useful property for showing correctness.

Insertion Sort Example

- **Initialization:** Before the start of the first iteration, j has been initialized to 2. The subarray $A[1 .. j\text{-}1]$ is just $A[1]$, which is trivially sorted.

```
INSERTION-SORT(A)
for j = 2 to A.length
    key = A[j]
    // Insert A[j] into sorted seq A[1 .. j-1]
    i = j – 1
    while i > 0 and A[i] > key
        A[i+1] = A[i]
        i = i – 1
    A[i+1] = key
```

Invariant: the subarray $A[1 .. j\text{-}1]$ consists of the elements originally in $A[1 .. j\text{-}1]$, but in sorted order

Insertion Sort Example

- **Maintenance:** (Sketch) By the property of the invariant, $A[1 .. j\text{-}1]$ is sorted.
  - Line 2 assigns $A[j]$ to *key*.
  - The **while** loop ensures that all array entries in $A[1 .. j\text{-}1]$ larger than *key* is shifted one place to the right.
  - Line 8 assigns *key* to location created by shifts.

INSERTION-SORT(A)

1. **for** $j = 2$ **to** $A.length$
2. $\quad key = A[j]$
3. $\quad$ // Insert $A[j]$ into sorted seq $A[1 .. j\text{-}1]$
4. $\quad i = j - 1$
5. $\quad$ **while** $i > 0$ and $A[i] > key$
6. $\quad\quad A[i+1] = A[i]$
7. $\quad\quad i = i - 1$
8. $\quad A[i+1] = key$

Inv: $A[i+2..j]$ sorted and $> key$.
$A[1..i] = A'[1..i]$ and $A[i+2..j] = A'[i+1..j\text{-}1]$
where A' is the array before the start of loop.

Insertion Sort Example

- **Termination:** Array length is n and after the final loop, j is incremented to n+1. From the invariant, we have $A[1 .. j\text{-}1]$ being sorted. Substituting j, the whole array is sorted.

```
INSERTION-SORT(A)
for j = 2 to A.length
    key = A[j]
    // Insert A[j] into sorted seq A[1 .. j-1]
    i = j – 1
    while i > 0 and A[i] > key
        A[i+1] = A[i]
        i = i – 1
    A[i+1] = key
```

Invariant: the subarray $A[1 .. j\text{-}1]$ consists of the elements originally in $A[1 .. j\text{-}1]$, but in sorted order

Pseudocode Convention

- In a **for** loop, we initially check the loop invariant immediately after loop-counter is initialized (j = 2 in insertion sort)
- Loop counter is changed at the end of the loop and retains its value after the loop terminates ($j = n+1$ in insertion sort)

```
INSERTION-SORT(A)
for j = 2 to A.length
    key = A[j]
    // Insert A[j] into sorted seq A[1 .. j-1]
    i = j –1
    while i > 0 and A[i] > key
        A[i+1] = A[i]
        i = i –1
    A[i+1] = key
```

- We use = for variable assignment and == for equality test, but older parts of the notes use ← for assignment and = for equality.

```
INSERTION-SORT(A)
for j = 2 to A.length
    key = A[j]
    // Insert A[j] into sorted seq A[1 .. j-1]
    i = j –1
    while i > 0 and A[i] > key
        A[i+1] = A[i]
        i = i –1
    A[i+1] = key
```

Reasoning About Iterative Algorithm

Recap

- Invent an invariant
- Show invariant true at initialization, is correctly maintained, and implies correctness with termination condition.

Analysis and Design of Algorithms

NUS National University of Singapore

CS3230

Reasoning About Algorithms

- Invariant and Iterative algorithms
- **Recursive algorithms**

Prof. Lee Wee Sun

Binary Search

Problem: Determine whether a number x is present in a *sorted* array $A[1 \ldots N]$

```
BINARY-SEARCH (A, a, b, x)
if  a > b then
    return false
else
    mid = ⌊(a+b)/2⌋
if x == A[mid] then
    return true
if x < A[mid] then
    return BINARY-SEARCH (A, a, mid-1, x)
else
    return BINARY-SEARCH (A, mid+1, b, x)
```

Correctness of Recursive Algorithm

Usually use **mathematical induction** on size of problem

```
BINARY-SEARCH (A, a, b, x) ▷ A[a . . b]
if    a > b then
   return false
else mid = ⌊(a+b)/2⌋
if x == A[mid] then
   return true
if x < A[mid] then
   return BINARY-SEARCH (A, a, mid-1, x)
else
   return BINARY-SEARCH (A, mid+1, b, x)
```

Base case: array size $n = b - a + 1 = 0$

The array is empty, so $a = b + 1$

The test $a > b$ succeeds and the algorithm correctly returns false

Inductive step:

Array size $n = b - a + 1 > 0$

Inductive hypothesis:

Assume **BINARY-SEARCH** (A, a', b', x) returns the correct value for all j such that $0 \le j \le n-1$ where $j = b' - a' + 1$.

The algorithm first calculates $mid = \lfloor (a+b)/2 \rfloor$, thus $a \le mid \le b$.

If $x == A[mid]$, clearly $x \in A[a..b]$ and the algorithm correctly returns true.

```
BINARY-SEARCH (A, a, b, x) ▷ A[a . . b]
if    a > b then
   return false
else mid = ⌊(a+b)/2⌋
if x == A[mid] then
   return true
if x < A[mid] then
   return BINARY-SEARCH (A, a, mid-1, x)
else
   return BINARY-SEARCH (A, mid+1, b, x)
```

If $x < A[mid]$,
x is in $A[a..b]$ if and only if
$x \in A[a..mid\text{-}1]$.

By the *inductive hypothesis*, **BINARY-SEARCH** $(A, a, mid\text{-}1, x)$ will return the correct value since
$0 \leq (mid\text{-}1) - a + 1 \leq n\text{-}1$.

```
BINARY-SEARCH (A, a, b, x) ▷ A[a . . b]
if    a > b then
   return false
else mid = ⌊(a+b)/2⌋
if x = A[mid] then
   return true
if x < A[mid] then
   return BINARY-SEARCH (A, a, mid-1, x)
else
   return BINARY-SEARCH (A, mid+1, b, x)
```

The case $x > A[mid]$ is similar

Reasoning About Recursive Algorithm

Recap

- Use strong induction
- Prove base cases
- Show algorithm works correctly assuming algorithm works correctly for all smaller cases